

C++ Systems Track · G0 Native Toolchain & Machine Boundary

C++ Systems Track

August 26, 2026

Contents

1	Part 0 · 统一主线与世界观	3
1.1	0.1 G0 的根本问题	3
1.2	0.2 源码世界与二进制世界	3
2	Part 1 · 编译流水线与分离编译	4
2.1	1.1 第一原则: C++ Program 不等于 .cpp 文件	4
2.2	1.2 Compiler Driver	4
2.3	1.3 Preprocessor 与 Textual Inclusion	4
2.4	1.4 Translation Unit (TU)	4
2.5	1.5 Declaration vs Definition	4
2.6	1.6 声明即足以编译调用方代码	4
2.7	1.7 Header 的工程定位	4
3	Part 2 · 目标文件与二进制结构	5
3.1	2.1 “Object”概念辨析	5
3.2	2.2 Object File (.o) 的内部组成	5
3.3	2.3 平台二进制格式	5
3.4	2.4 Segment 与 Section	5
3.5	2.5 Symbol 与 Undefined Symbol	5
3.6	2.6 Relocation (重定位)	5
3.7	2.7 Name Mangling	6
4	Part 3 · 静态库、动态库与加载器	6
4.1	3.1 静态库 (.a) 与符号按需抽取	6
4.2	3.2 动态库 (.dylib / .so)	6
4.3	3.3 Dynamic Linking 的两个阶段	6
4.4	3.4 Linker vs Loader	6
4.5	3.5 macOS @rpath	6
4.6	3.6 五层错误诊断模型	6
5	Part 4 · 接口契约: API、ABI 与 FFI	7
5.1	4.1 API 与 ABI	7
5.2	4.2 ABI 的组成维度	7
5.3	4.3 Calling Convention (以 Apple ARM64 为例)	7
5.4	4.4 Alignment, Padding 与 Struct Layout	7
5.5	4.5 Source Compatibility ≠ ABI Compatibility	7
5.6	4.6 extern “C” 与 Language Linkage	7
5.7	4.7 C ABI 作为跨语言边界	7
5.8	4.8 Opaque Handle 与跨边界所有权	8

6	Part 5 · 机器运行时：进程、调用栈与寄存器	8
6.1	5.1 Program vs Process 与虚拟地址空间	8
6.2	5.2 Stack, Stack Frame 与 Automatic Storage Duration	8
6.3	5.3 SP, LR(x30) 与 FP(x29)	8
6.4	5.4 Leaf Functions 与返回地址保存	8
6.5	5.5 Prologue 与 Epilogue	8
6.6	5.6 Caller-Saved vs Callee-Saved	9
6.7	5.7 Apple Silicon ARM64 核心寄存器速查	9
6.8	5.8 Register Spill 与 As-if 规则	9
6.9	5.9 Recursion 与 Stack Overflow	9
6.10	5.10 悬挂指针与未定义行为 (UB)	9
7	Part 6 · 构建系统、诊断模型与核心铁律	9
7.1	6.1 构建系统的本质	9
7.2	6.2 原生工具链的四层世界	10
7.3	6.3 诊断与性能习惯	10
7.4	6.4 G0 自测清单	10
7.5	6.5 十条不可动摇的核心铁律	10

Version: 1.0 (Frozen Review Baseline)

Scope: Compilation Model / Object Files / Linking / Libraries / ABI / Machine State & Stack

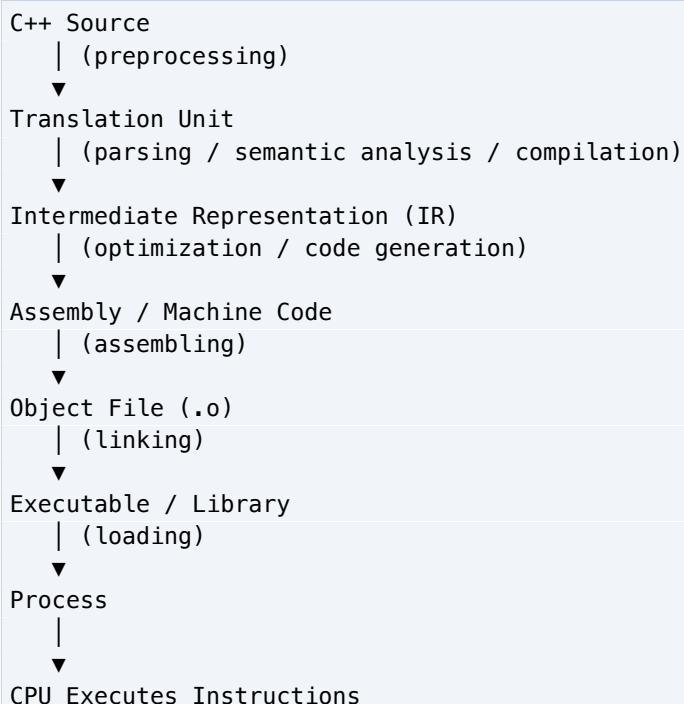
Target Environment: macOS · Apple Silicon · Clang/Apple Clang

Purpose: 作为进入 G1 C++ Object Model 前的长期系统底座复习与推理手册

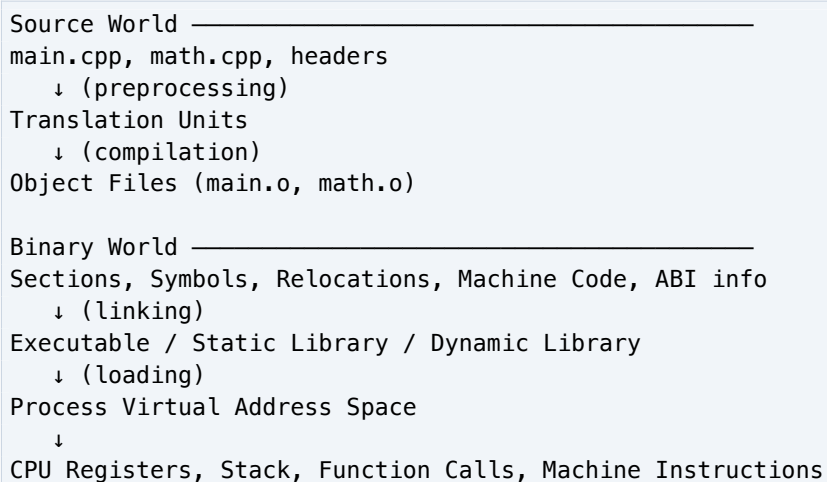
1 Part 0 · 统一主线与世界观

1.1 0.1 G0 的根本问题

一段 C++ 源代码究竟怎样变成 CPU 正在执行的机器指令？



1.2 0.2 源码世界与二进制世界



2 Part 1 · 编译流水线与分离编译

2.1 1.1 第一原则：C++ Program 不等于 .cpp 文件

一个 .cpp 只是程序源代码的一部分。传统 C++ 编译模型的基本思想是 Separate Compilation（分离编译）：

```
main.cpp    —(compile)—> main.o    \
network.cpp —(compile)—> network.o \
storage.cpp —(compile)—> storage.o —(Linker)—> Executable
parser.cpp  —(compile)—> parser.o  /
```

工程思考原点：遇到头文件、符号冲突、ODR、模板膨胀或增量重构问题时，首先排查：该问题是否源于 Separate Translation + Linking？

2.2 1.2 Compiler Driver

clang++ 首先是一个 Compiler Driver（编译器驱动器），负责调度协调整条原生工具链：

```
clang++ driver
├── Preprocessor
├── Compiler Frontend
├── Optimizer / Backend
├── Assembler
└── Linker
```

2.3 1.3 Preprocessor 与 Textual Inclusion

执行 `clang++ -E main.cpp -o main.i` 只执行预处理阶段。

核心机制：传统 `#include` 是 Textual Inclusion（文本包含），非 Module Import。

`#include` ≠ Python `import` ≠ Rust `use` ≠ Zig `@import`。

2.4 1.4 Translation Unit (TU)

定义：Translation Unit = 一个 .cpp + 递归 `#include` 的内容 + 宏展开等预处理结果。

分离特性：编译器在编译当前 TU 时，并不天然知道其他 TU 的完整实现。

2.5 1.5 Declaration vs Definition

Declaration（声明）：`int add(int, int);` □ 接口契约/承诺（Contract / Promise），告诉编译器符号名、参数类型与返回值类型。

Definition（定义）：`int add(int a, int b) { return a + b; }` □ 实体兑现（Implementation / Fulfillment），生成具体的机器码或分配实际存储。

2.6 1.6 声明即足以编译调用方代码

`clang++ -c main.cpp -o main.o` 在仅有 `add` 声明时即可编译成功。编译器依据声明和目标平台 ABI，已足够确定传参寄存器与返回方式，并生成调用指令。

2.7 1.7 Header 的工程定位

Header 是共享 Declaration 的 Source of Truth。

工程实践：`math.cpp` 必须 `#include "math.hpp"`，使编译器在同一个 TU 内尽早发现声明与定义不匹配。

3 Part 2 · 目标文件与二进制结构

3.1 2.1 “Object”概念辨析

C++ Object: `int x = 42;` (语言语义层实体, 具有生命周期与类型)。

Object File: `main.o` (编译器/链接器层面的目标文件, 可重定位二进制工件)。

Object Code: 面向目标架构的机器指令代码。

3.2 2.2 Object File (.o) 的内部组成

.o 是结构化的二进制文件, 而非裸机器指令:

```
Object File (.o)
├─ Machine Instructions (__text)
├─ Sections (__const, __data, etc.)
├─ Symbol Table (Defined & Undefined)
├─ Relocation Records
├─ Constants / Metadata
└─ Optional Debug Information
```

3.3 2.3 平台二进制格式

macOS (Apple Silicon) □ Mach-O 64-bit object arm64

Linux □ ELF

Windows □ PE / COFF

3.4 2.4 Segment 与 Section

Section (节): 编译器与链接器组织二进制内容的细粒度逻辑分类 (如 `__text`, `__const`, `__cstring`, `__data`)。

Segment (段): 操作系统加载器 (Loader) 与虚拟内存映射的较大权限管理单位 (如 `__TEXT` 为 R-X, `__DATA` 为 RW-)。

```
__TEXT (Segment: Read + Execute)
├─ __text (Machine instructions)
├─ __const (Read-only constants)
└─ __cstring (String literals)
```

3.5 2.5 Symbol 与 Undefined Symbol

Symbol: Linker 能识别和解析的命名二进制实体。

`nm -C main.o:`

T: 当前 Object 提供的 Text/Code 定义。

U: Undefined Symbol (当前 Object 使用了该符号, 但自身未提供定义)。

合法性: .o 中存在 U 符号完全合法, 只要在最终静态链接或动态加载阶段能解析即可。

3.6 2.6 Relocation (重定位)

编译单个 TU 时, 外部函数的最终运行时虚拟地址未知。编译器会生成占位调用指令, 并在 Relocation 表中记录修补位置 (Offset) 与目标符号。链接器在确定所有符号地址后执行 Patch。

3.7 2.7 Name Mangling

为支持重载、命名空间与模板，C++ 编译器将类型信息编码进符号名称：

C++ 源码：int add(int, int)

Itanium ABI 符号：_Z3addii

nm -C：执行 Demangling，将符号还原为人类可读源码形式。

4 Part 3 · 静态库、动态库与加载器

4.1 3.1 静态库 (.a) 与符号按需抽取

本质：一组.o 文件的归档包 (Archive)。

抽取机制：静态链接器默认依据未解析符号 (Unresolved Symbols) 的需求按需抽取.o 成员，而非机械全量拷贝。

独立性：静态链接完成后，删除.a 文件不影响已生成的可执行文件运行。

4.2 3.2 动态库 (.dylib / .so)

动态库保持为独立的 Binary Image。可执行文件在链接时仅记录依赖声明，不将其实现代码合并至自身。

4.3 3.3 Dynamic Linking 的两个阶段

Link Time (构建期)：静态链接器检查动态库能否提供所需符号，并在可执行文件中写入依赖记录。

Load Time / Runtime (启动/运行期)：操作系统动态链接器 (macOS 为 dyld) 映射 dylib 到进程空间、重定位并绑定符号。

4.4 3.4 Linker vs Loader

Linker (构建期)：符号解析、重定位修补、Section 布局、二进制产物生成。

Loader / dyld (运行期)：映射镜像、加载依赖库、运行时符号绑定、执行全局初始化。

4.5 3.5 macOS @rpath

@rpath/libmath.dylib：运行时动态库搜索路径机制。

报错 Library not loaded: @rpath/... 属于 Load-Time Failure，非编译/链接错误。

4.6 3.6 五层错误诊断模型

- | | |
|--------------------------|--|
| 1. Preprocessing Failure | —> 缺少头文件 / 宏展开错误 |
| 2. Compilation Failure | —> 语法错误 / 类型不匹配 |
| 3. Linking Failure | —> Undefined symbol / Duplicate symbol |
| 4. Load Failure | —> 找不到 dylib / @rpath 配置错误 |
| 5. Runtime Failure | —> 段错误 (Segfault) / 逻辑异常 / UB |

5 Part 4 · 接口契约：API、ABI 与 FFI

5.1 4.1 API 与 ABI

API (Application Programming Interface): 源码层契约, 决定程序员如何调用接口。

ABI (Application Binary Interface): 二进制层契约, 决定已编译的机器码如何在寄存器和内存层正确协同。

5.2 4.2 ABI 的组成维度

ABI

- Calling Convention (参数/返回值寄存器、栈对齐、寄存器保存约定)
- Data Layout (sizeof, alignof, padding, 成员偏移)
- Symbol Convention (Name Mangling 规范)
- Runtime ABI (Exceptions unwinding, RTTI, vtable 结构)
- Dynamic Linking Contract (动态加载与符号绑定协议)

5.3 4.3 Calling Convention (以 Apple ARM64 为例)

参数传递: 通用整型/指针使用 $x0 - x7$ (32 位视图为 $w0 - w7$), 浮点使用 $v0 - v7$ 。

返回值: 放置于 $x0 / v0$ 。

指令: BL 跳转并链接 (写入返回地址到 LR), RET 读取 LR 返回。

5.4 4.4 Alignment, Padding 与 Struct Layout

Alignment: 类型起始地址必须满足的对齐边界约束 (如 $\text{alignof}(T) == 4$ 要求 $\text{address} \% 4 == 0$)。

Padding & Tail Padding: 编译器为满足成员对齐及数组元素连续对齐而插入的填充间隙。

```
struct Packet {
    char tag;      // 1 byte | offset 0
                  // 3 bytes padding (offset 1-3)
    int value;     // 4 bytes | offset 4-7
    short code;    // 2 bytes | offset 8-9
                  // 2 bytes tail padding (offset 10-11)
}; // sizeof(Packet) == 12, alignof == 4
```

5.5 4.5 Source Compatibility ≠ ABI Compatibility

结构体增加字段或重排, 源码重新编译可能完全正常 (源码兼容); 但旧二进制若按旧偏移寻址, 会导致内存访问错位 (ABI 破坏)。

5.6 4.6 extern "C" 与 Language Linkage

extern "C" 指定 C language linkage。在常见 ABI/toolchain 上, 其外部符号采用 C linkage 的命名约定, 而不是普通 C++ 基于重载的 mangling。函数体内部仍然享有完全的 C++ 语言语义 (RAII、模板、异常等)。

5.7 4.7 C ABI 作为跨语言边界

C ABI 相对简单、广泛支持、长期稳定性较强, 且避免暴露 C++ class/template/exception/stdlib 等复杂语言特有 ABI, 因此成为 C++、Rust、Zig、Python 等语言 FFI 交互的通用桥梁。但 C ABI 自身依然受调用约定、结构体返回规则、对齐与平台 ABI 约束。

5.8 4.8 Opaque Handle 与跨边界所有权

Opaque Handle（不透明句柄）：稳定 FFI 接口的首选范式，向外部隐藏具体内存布局：

```
typedef struct engine_handle engine_handle;
engine_handle* engine_create(void);
void engine_destroy(engine_handle*);
```

Ownership 契约：遵循“谁分配谁销毁”原则，禁止跨运行时混合 malloc/free。

Exception 屏障：严禁 C++ 异常穿透 C ABI 边界，边界函数应标记 noexcept 并转换为错误码返回。

6 Part 5 · 机器运行时：进程、调用栈与寄存器

6.1 5.1 Program vs Process 与虚拟地址空间

Program：磁盘上的静态二进制工件。

Process：运行中的程序实例，拥有独立的虚拟地址空间、线程栈、文件描述符与内核资源。

6.2 5.2 Stack, Stack Frame 与 Automatic Storage Duration

Stack：支撑函数嵌套调用的 LIFO 内存结构。

Stack Frame：单次函数调用的栈状态记录。

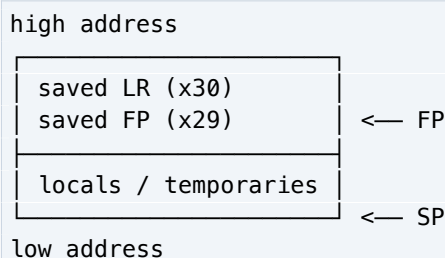
Automatic Storage Duration：局部变量在语言语义层具有自动存储期，但在机器层编译器可通过寄存器化（Registerize）、常量折叠或优化消除而不占用物理栈槽。

6.3 5.3 SP, LR(x30) 与 FP(x29)

SP (Stack Pointer)：指向当前栈顶，ARM64 下要求 16 字节对齐。

LR / x30 (Link Register)：保存子函数执行完毕后的返回地址。

FP / x29 (Frame Pointer)：记录当前栈帧基址，形成用于 Debugger 和 Unwinding 的 Backtrace 链。



6.4 5.4 Leaf Functions 与返回地址保存

Leaf Function（叶子函数）：不调用任何其他函数的末端函数。由于 LR 不会被子调用覆盖，通常不需要分配完整栈帧或保存 LR。

Non-Leaf Function：如果函数在调用其他函数后仍需返回原 caller，就必须 preserve return address；常见实现是在 stack frame 中保存 LR，但编译器也可以采用其他 ABI 合法方式（如 tail call 等情形可能无需普通保存/恢复）。

6.5 5.5 Prologue 与 Epilogue

Prologue：函数入口处建立栈帧（保存 FP/LR、调整 SP、设置新 FP）。

Epilogue：函数退出前清理栈帧（恢复寄存器、恢复 SP、ret 返回）。

6.6 5.6 Caller-Saved vs Callee-Saved

Caller-Saved: 调用方若在调用后仍需使用该寄存器, 需自行在调用前保存。

Callee-Saved: 被调用方若修改这些寄存器, 必须在退出前恢复其原值。

6.7 5.7 Apple Silicon ARM64 核心寄存器速查

寄存器	约定类别	主要用途与角色
x0 - x7	Caller-Saved	参数传递 / 返回值传递 (低 32 位为 w0 - w7)
x9 - x15	Caller-Saved	临时通用计算寄存器
x19 - x28	Callee-Saved	长期局部变量与跨调用状态保存
x29 (FP)	Callee-Saved	栈帧指针 (Frame Pointer)
x30 (LR)	Caller-Saved (特殊)	链接寄存器 (Link Register, 保存返回地址)
SP	专职寄存器	栈指针 (Stack Pointer, 需保持 16 字节对齐)
v0 - v31	混合	浮点与 SIMD 向量寄存器 (d0-d31 / s0-s31)

6.8 5.8 Register Spill 与 As-if 规则

Spill: 当 register pressure、calling convention、debug/codegen strategy (如 -O0) 等使某个值无法或不适合继续保存在寄存器中时, 编译器可能把它 spill 到 stack。

As-if Rule: 编译器只要保证程序的可观测行为不变, 即可对机器码进行任意优化。源码描述的是计算语义, 而不是最终机器执行模型。

6.9 5.9 Recursion 与 Stack Overflow

深层递归持续分配栈帧。当 调用深度 × 栈帧大小 > 可用栈内存时, 将触发 Stack Overflow (macOS 下表现为 EXC_BAD_ACCESS)。

6.10 5.10 悬挂指针与未定义行为 (UB)

```
int* bad() {  
    int x = 42;  
    return &x; // x lifetime 结束  
}
```

函数返回时仅调整 SP, 原栈内存的 bits 不会立即清零。但内存 bits 还在 ≠ C++ 对象仍然存活。读取悬挂指针属于 Undefined Behavior。

7 Part 6 · 构建系统、诊断模型与核心铁律

7.1 6.1 构建系统的本质

CMake、Ninja、Make、Zig Build 的核心是管理 Artifact Derivation Graph (构建工件的有向无环依赖图 DAG)。

7.2 6.2 原生工具链的四层世界

1. Language World → Types, Declarations, Definitions, Scope, Lifetime
2. Compiler World → TU, AST, IR, Optimizations, Code Generation
3. Binary World → Object Files, Symbols, Relocations, ABI, Libraries
4. Machine World → Process, Virtual Address Space, Stack, Registers, CPU Instructions

7.3 6.3 诊断与性能习惯

诊断分层：不要泛称“编译报错”，先精准定位发生在五层中的哪一层（预处理、编译、链接、加载还是运行）。

性能观察：源码写了变量不等于内存中一定分配；性能调优必须观察汇编、Profiler 与 Cache 行为。

7.4 6.4 G0 自测清单

- 能否清晰解释.cpp 到 CPU 执行的完整阶段与各个中间工件？
- 为什么只有 Declaration 也能成功生成包含调用的.o 文件？
- Relocation 到底解决了单文件编译时的什么核心问题？
- .a 静态库与 .dylib 动态库在链接和加载时有何本质区别？
- 为什么 public struct 调整字段顺序属于破坏 ABI？
- ARM64 的 LR 与 FP 在非叶子函数调用中分别承担什么职责？
- 为什么局部变量退出作用域后原地址可能还能读出旧数据，但这依然是严重 UB？

7.5 6.5 十条不可动摇的核心铁律

1. C++ 程序不等于.cpp：原生程序是 Source → TU → Object → Link → Load → Process 的系统化结果。
2. Translation Unit 是传统 C++ 分离编译的核心独立单元。
3. Declaration 提供接口契约，Definition 真正提供实体实现。
4. Object File 是结构化的二进制工件，包含机器码、Sections、符号表与重定位记录。
5. Linker 的核心职责是符号解析（Symbol Resolution）与地址重定位（Relocation）。
6. 静态库是.o 归档包；动态库是独立的运行时镜像。
7. API 是源码级调用契约，ABI 是机器码级协作契约。
8. 源码变量、对象生命周期与物理内存栈槽是完全解耦的概念。
9. 函数调用在底层统一落地为寄存器传参、栈对齐、返回地址保存与 ABI 规范。
10. 源码仅描述程序语义，不等于机器执行模型；系统级分析必须能够下沉至二进制与硬件层。